PicoBot AGV — Software System Diagram

IDP Group M108 · Software subteam · how each physical connection maps to code in sw/ (V3.5) · PicoBot circuit schematic

SIGNAL TYPE

- Digital in
- PWM out
- Analogue (ADC)
- Digital out
- I²C bus
- source file → class

INPUTS — SENSORS

4× Line tracker DFRobot SEN0017
3V3 · TTL: HIGH over white (LINE_WHITE_IS_HIGH)
`drivers/line_sensor.py → LineSensorArray`

- **Line L** inner — follow
- **Line R** inner — follow
- **Turn L** outer — junctions
- **Turn R** outer — junctions

Hall sensor DRV5055 (U1)
3V3 · analogue · reads box magnet pole at gripper
`drivers/magnet.py → MagnetSensor`

IR receiver TSDP43xx (U2)
3V3 · traffic light at U1→U2 · read_u16 < 1000 = green
`nav/traffic_light.py → TrafficLight`

ToF distance VL53L0X (rear)
I²C0 @ 400 kHz · bay full ≤ 215/220 mm, else empty
`drivers/distance.py → libs/VL53L0X`

Start/stop button SW1
via ACPL-847 opto ch.1 · edge IRQ latched, 30 ms debounce
`drivers/button.py → Button`

CONTROLLER

PicoBot

Raspberry Pi Pico (RP2040) · MicroPython
main.py auto-runs at power-up

DI	GP18 · 24	
DI	GP19 · 25	
DI	GP22 · 29	
DI	GP17 · 22	
ADC1	GP27 · 32	
ADC2	GP28 · 34	
SDA	GP8 · 11	
SCL	GP9 · 12	
IRQ	GP20 · 26	
	GPxx · header pin	
	GP4 · 6	DIR
	GP5 · 7	PWM
	GP6 · 9	PWM
	GP7 · 10	DIR
	GP15 · 20	PWM
	GP13 · 17	PWM
	GP26 · 31	DO
	GP16 · 21	DO
	GP21 · 27	DO

POWER

- 12 V — battery in
- 12 V — → Motor Driver 2
- 6 V5 — → servos
- 3V3 — → sensors, opto

OUTPUTS — DRIVERS & ACTUATORS

Left drive motor Motor 3
Geared DC 1271-12-43 · 12 V · Motor Driver 2 ch.1
`drivers/drive.py → Drive → motor.py → Motor`
dir pin + PWM 1 kHz · speed −100…100 · trim 0.92

Right drive motor Motor 4
Geared DC 1271-12-43 · 12 V · Motor Driver 2 ch.2
`drivers/drive.py → Drive → motor.py → Motor`
reverse_right=True (mirrored mount) · trim 1.0

Grabber servo SER0038 · Servo 1
6 V5 rail · PWM 50 Hz · open 70° / closed −15°
`drivers/servo.py → Servo.set_Grabber_angle`

Lifter servo SER0038 · Servo 2
6 V5 rail · PWM 50 Hz · down −5° / carry 45° / idle 60°
`drivers/servo.py → Servo.set_Lifter_angle`

Indicator LEDs via ACPL-847 opto
Pico never drives LEDs directly (M1 build spec)
`drivers/indicators.py → FlashingLamp, PoleIndicator`

- **Amber beacon** 2 Hz flash while moving
- **Red** steady = North pole
- **Green** steady = South pole

SOFTWARE STACK — sw/

calls flow downwards; only drivers/ and libs/ touch the hardware

config.py
every GPxx pin
speeds & steer
servo angles
timings (ms)
bay thresholds
mission settings
edit here, not in the logic

main.py — **main()** builds every object from config, calibrates magnet, waits for button press, then **run()** loops every 5 ms (LOOP_PERIOD_MS):
stop button? → beacon.update() → delivery bay / junction (collect · check bay · turn) → traffic light → follower.tick() · never blocks · finally: drive.stop()

nav/

mission.py	navigator.py	map.py	junction.py	follower.py	bay.py	traffic_light.py
Mission	State	JUNCTIONS	JunctionDetector	LineFollower	BayDetector	TrafficLight
collect/deliver/home	heading → turn	graph + path_find	debounced count	steer per tick	full / empty	wait at U1→U2

drivers/

button.py	line_sensor.py	magnet.py	distance.py	drive.py	motor.py	servo.py	indicators.py
Button	LineSensorArray	MagnetSensor	build_rear()	Drive	Motor	Servo	PoleIndicator
IRQ latch	4 × Pin.IN	ADC, deadband	VL53L0X / NoSensor	move/rotate/u_turn	dir + PWM	angle → duty	+ FlashingLamp

libs/ + HAL — **libs/VL53L0X** ToF driver · MicroPython **machine**: Pin · PWM · ADC · I2C · **utime**: ticks_ms · sleep_ms · bench tests per device in **tests/**

Pin labels read "GPxx · header pin": GPxx is the machine.Pin number used in config.py; the header pin is the PicoBot pin number on the circuit schematic.

Generative AI declaration: all algorithmic flows were devised by the group. AI was used for repetitive changes, formatting, debugging and commenting.

Content: IDP Group M108 (software subteam) · Layout: Claude (Anthropic)

Repository: `github.com/blakehrobinson-design/IDP-repo/tree/rear_bay_detect/V3.5/sw`